Оглавление

Введение 4

1. Анализ предметной области 6

1.1. Назначение образовательных тестов 6

1.2. Пользователи системы 6

1.3. Основные сценарии использования 7

2. Требования к приложению 8

2.1. Функциональные требования 8

2.2. Нефункциональные требования 9

3. Архитектура клиент-серверного приложения 12

3.1. Обоснование выбора клиент-серверной архитектуры 12

3.2. Общая структура приложения 12

3.3. Взаимодействие компонентов 14

3.4. Применение методологии двенадцати факторов 15

4. Выбор программного стека 17

4.1. Frontend: React, React Router, Redux Toolkit, CSS Modules 17

4.2. Backend: Node.js, Express.js 17

4.3. База данных: PostgreSQL 18

4.4. Инфраструктура: Docker, Docker Compose, Nginx 18

4.5. Система контроля версий: Git 19

4.6. Обоснование выбора технологий 19

5. Проектирование базы данных 21

5.1. Основные сущности 21

5.2. Связи между сущностями 22

5.3. Хранение вопросов разных типов 23

6. Разработка клиентской части 24
6.1. Структура React-приложения 24
6.2. Маршрутизация страниц 25
6.3. Главная страница 25
6.4. Страница тестов 26
6.5. Компонент прохождения теста 26
6.6. Конструктор вопросов 27
6.7. Работа с состоянием через Redux 27
7. Разработка серверной части 29
7.1. Структура backend-приложения 29
7.2. REST API 30
7.3. Реализация CRUD-операций 30
7.4. Авторизация и аутентификация через JWT 31
7.5. Проверка ролей пользователей 31
7.6. Работа с PostgreSQL 32
8. Тестирование приложения 34
8.1. Регистрация и авторизация 34
8.2. Проверка основных пользовательских сценариев 36
8.2.1 Просмотр списка тестов 36
8.2.2 Прохождение теста 36
8.2.3 Повтор ошибок 37
8.2.4 Создание теста 38
8.2.5 Добавление вопроса 38
8.3. Модульное и компонентное тестирование 40
8.4. Фаззинг-тестирование API 41

9. Размещение проекта в Git и контейнеризация 42

10. Контейнеризация и развёртывание 43

10.1. Dockerfile клиентской части 43

10.2. Dockerfile серверной части 43

10.3. Docker Compose 44

10.4. PostgreSQL-контейнер 44

10.5. Nginx как reverse proxy 45

10.6. Развёртывание в облаке 45

Заключение 46

Список используемой литературы 48

Введение

В условиях активного развития цифровых технологий и перехода образовательных процессов в онлайн-среду особую значимость приобретают инструменты автоматизации контроля знаний. Одним из наиболее распространённых и эффективных методов оценки уровня подготовки обучающихся являются тестовые системы. Они позволяют оперативно проверять знания, обеспечивают объективность оценки и удобны для масштабирования на большое количество пользователей.

Современные образовательные платформы предъявляют высокие требования к гибкости и интерактивности подобных систем. В частности, актуальной задачей является создание инструментов, позволяющих преподавателям самостоятельно формировать тестовые задания, управлять структурой тестов и анализировать результаты прохождения. Это определяет необходимость разработки специализированных программных решений — интерактивных конструкторов образовательных тестов.

Целью данной курсовой работы является разработка клиент-серверного приложения «Интерактивный конструктор образовательных тестов», обеспечивающего создание, редактирование и прохождение тестов с поддержкой различных типов вопросов и ролевой модели пользователей.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области;
- определить требования к системе;
- выбрать архитектуру программного решения и описать её с использованием UML;
- обосновать выбор программного стека для реализации fullstack-приложения;
- реализовать клиентскую и серверную части системы;
- обеспечить механизм аутентификации и авторизации пользователей;
- реализовать взаимодействие с базой данных и заполнить её тестовыми

данными;

- реализовать ролевую модель и выполнить валидацию входных данных;
- провести тестирование приложения, включая фаззинг-тестирование;
- организовать хранение исходного кода в системе контроля версий;
- выполнить контейнеризацию и развертывание приложения в облачной среде;
- подготовить презентационные материалы.

Объектом исследования является процесс автоматизации контроля знаний обучающихся с использованием тестовых систем.

Предметом исследования выступают методы и средства разработки веб-приложений, реализующих функциональность создания и прохождения образовательных тестов.

В рамках работы используется клиент-серверная архитектура, а также современные технологии веб-разработки, включая JavaScript, библиотеку React для создания пользовательского интерфейса, платформу Node.js для серверной части, систему управления базами данных PostgreSQL, а также инструменты контейнеризации Docker и систему контроля версий Git. При разработке учитываются принципы методологии двенадцати факторов, направленные на создание масштабируемых и переносимых приложений.

Практическая значимость работы заключается в создании программного продукта, который может быть использован в образовательных учреждениях для организации тестирования и контроля знаний обучающихся.

1. Анализ предметной области

1.1. Назначение образовательных тестов

Образовательные тесты являются распространённым инструментом контроля знаний обучающихся. Они позволяют оценить уровень усвоения материала, выявить пробелы и обеспечить объективность проверки за счёт заранее заданных критериев.

Тестирование широко применяется для текущего, промежуточного и итогового контроля как в очном, так и в дистанционном обучении. Современные системы требуют интерактивных инструментов, позволяющих преподавателям создавать и редактировать тесты, а студентам — проходить их и получать результат.

Разрабатываемое приложение предназначено для автоматизации этих процессов, обеспечивая создание тестов, их прохождение и анализ результатов.

1.2. Пользователи системы

В разрабатываемой системе предусмотрены три основных типа пользователей: студент, преподаватель и администратор. Каждый из них обладает определённым набором прав и выполняет свои функции.

Студент является основным пользователем системы. Он проходит образовательные тесты: после регистрации и входа получает доступ к списку тестов, выбирает нужный и отвечает на вопросы, после чего получает результат.

Преподаватель отвечает за создание и сопровождение тестовых материалов. Он может создавать новые тесты, добавлять и редактировать вопросы, а также удалять устаревшие задания. Это позволяет гибко управлять содержанием тестов без изменения кода приложения.

Администратор выполняет функции управления системой. Он обладает расширенными правами, включая контроль данных и пользователей, а также обеспечивает корректность работы ролевой модели.

Разделение пользователей по ролям необходимо для ограничения доступа к функциональности системы и защиты данных. Например, студент не имеет доступа к созданию и редактированию тестов, тогда как преподаватель и администратор обладают такими возможностями.

1.3. Основные сценарии использования

Основные сценарии использования отражают типовые действия пользователей в системе.

Пользователь может зарегистрироваться и выполнить вход в систему, после чего получает доступ к функциональности в зависимости от своей роли.

Студент выбирает тест из списка, проходит его, отвечая на вопросы, и после завершения получает результат. При необходимости он может повторить тест или вернуться к списку доступных тестов.

Преподаватель создаёт тесты, задаёт их параметры и добавляет вопросы различных типов: с одним или несколькими правильными ответами, а также на сопоставление. Он также может редактировать и удалять тесты и вопросы.

После прохождения теста система сохраняет и отображает результат.

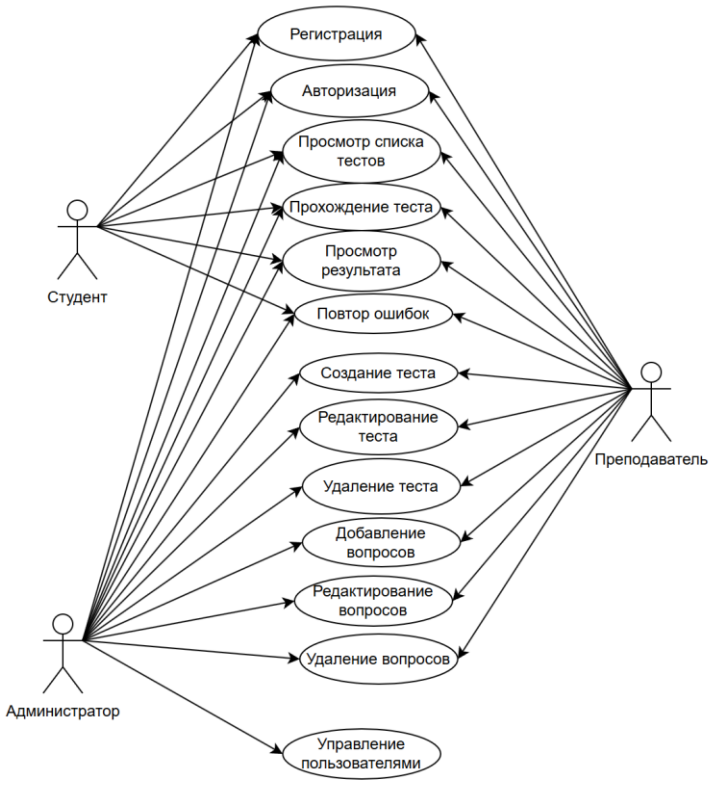

Рисунок 1 — UML-диаграмма вариантов использования

2. Требования к приложению

Разрабатываемое приложение «Интерактивный конструктор образовательных тестов» должно обеспечивать создание, редактирование и прохождение тестов. При формировании требований учитываются клиент-серверная архитектура, разграничение доступа пользователей и возможность развёртывания в контейнеризированной среде.

2.1. Функциональные требования

Функциональные требования определяют основные возможности приложения.

Система должна обеспечивать регистрацию пользователей и вход в систему. После авторизации пользователь получает доступ к функциям в соответствии со своей ролью.

Студент должен иметь возможность просматривать список тестов, выбирать тест и проходить его, отвечая на вопросы. После завершения система рассчитывает результат и отображает его пользователю.

Преподаватель должен иметь возможность создавать тесты, задавать их параметры, добавлять вопросы, а также редактировать и удалять их. В системе поддерживаются вопросы с одним правильным ответом, несколькими правильными ответами и на сопоставление.

Серверная часть должна обеспечивать выполнение CRUD-операций для пользователей, тестов, вопросов и результатов. Клиентская часть взаимодействует с сервером через API и отображает полученные данные.

Результаты прохождения тестов должны сохраняться в базе данных для последующего анализа.

Основные функциональные требования представлены в таблице 1.

Таблица 1 — Функциональные требования к приложению

№	Требование	Описание
1	Регистрация пользователя	Пользователь может создать учётную запись
2	Авторизация пользователя	Пользователь может войти в систему по email и паролю
3	Просмотр списка тестов	Пользователь может получить список доступных тестов

4	Прохождение теста	Студент может отвечать на вопросы и завершать тест
5	Расчёт результата	Система рассчитывает количество правильных ответов
6	Создание теста	Преподаватель может создать новый тест
7	Редактирование теста	Преподаватель может изменять данные теста
8	Удаление теста	Преподаватель может удалить тест
9	Добавление вопросов	Преподаватель может добавлять вопросы разных типов
10	Работа с БД	Данные пользователей, тестов и результатов сохраняются в PostgreSQL

2.2. Нефункциональные требования

Нефункциональные требования определяют характеристики качества приложения и условия его эксплуатации.

Приложение должно иметь удобный и понятный пользовательский интерфейс, обеспечивающий быстрый доступ к основным функциям. Интерфейс должен корректно отображаться в современных браузерах и адаптироваться под различные размеры экранов.

Система должна обеспечивать стабильную работу при основных действиях пользователей, включая регистрацию, вход, создание тестов и прохождение тестирования.

Приложение должно быть переносимым и удобным для развёртывания за счёт использования Docker и Docker Compose, что позволяет запускать компоненты системы в контейнерах.

Также система должна соответствовать принципам методологии двенадцати факторов, включая использование переменных окружения для конфигурации, явное описание зависимостей и воспроизводимость запуска.

Основные нефункциональные требования представлены в таблице 2.

Таблица 2 — Нефункциональные требования к приложению

№	Требование	Описание
1	Удобство использования	Интерфейс должен быть понятным для студента и преподавателя
2	Надёжность	Приложение должно корректно обрабатывать ошибки пользователя
3	Переносимость	Приложение должно запускаться в Docker-контейнерах
4	Масштабируемость	Архитектура должна позволять развивать приложение

5	Сопровождаемость	Код должен быть разделён на клиентскую и серверную части
6	Конфигурируемость	Настройки должны задаваться через переменные окружения
7	Совместимость	Приложение должно работать в современных браузерах

2.3. Требования к безопасности

Так как приложение работает с пользовательскими данными, необходимо предусмотреть базовые меры безопасности.

В системе реализуется аутентификация пользователей с использованием email и пароля. После успешного входа сервер формирует токен доступа (JWT), который используется при последующих запросах.

Пароли пользователей хранятся в зашифрованном виде с применением хеширования, что снижает риск компрометации данных.

Доступ к функциональности приложения ограничивается в зависимости от роли пользователя. Студент имеет доступ только к прохождению тестов, тогда как преподаватель и администратор могут управлять тестами и вопросами.

Серверная часть должна выполнять валидацию входных данных, обрабатывая некорректные значения и предотвращая ошибки при работе API.

Конфиденциальные параметры, такие как строка подключения к базе данных и JWT-секрет, должны храниться в переменных окружения.

Основные требования к безопасности представлены в таблице 3.

Таблица 3 — Требования к безопасности

№	Требование	Описание
1	Аутентификация	Доступ к системе осуществляется после входа пользователя
2	Авторизация	Доступ к функциям определяется ролью пользователя
3	Хеширование паролей	Пароли не хранятся в открытом виде
4	Использование JWT	Для подтверждения сессии используется токен доступа
5	Валидация данных	Сервер проверяет входные данные перед обработкой
6	Защита конфигурации	Секретные параметры хранятся в переменных окружения
7	Ограничение доступа	CRUD-операции конструктора доступны только разрешённым ролям

2.4. Требования к ролям и доступу

В приложении используется ролевая модель доступа, обеспечивающая разделение возможностей пользователей и предотвращение выполнения запрещённых действий.

Студент может регистрироваться, входить в систему, просматривать тесты, проходить их и получать результат, но не имеет доступа к созданию и редактированию тестов.

Преподаватель управляет учебными материалами: создаёт тесты, добавляет, изменяет и удаляет вопросы, а также использует конструктор тестов.

Администратор обладает расширенными правами, включая функции преподавателя и дополнительные возможности управления системой, и имеет полный доступ к функциональности приложения.

Распределение прав доступа представлено в таблице 4.

Таблица 4 — Ролевая модель доступа

Действие	**Студент**	**Преподаватель**	**Администратор**
Регистрация	Да	Да	Да
Вход в систему	Да	Да	Да
Просмотр списка тестов	Да	Да	Да
Прохождение тестов	Да	Да	Да
Просмотр результата	Да	Да	Да
Создание тестов	Нет	Да	Да
Редактирование тестов	Нет	Да	Да
Удаление тестов	Нет	Да	Да
Добавление вопросов	Нет	Да	Да
Редактирование вопросов	Нет	Да	Да
Удаление вопросов	Нет	Да	Да
Доступ к конструктору	Нет	Да	Да

3. Архитектура клиент-серверного приложения

3.1. Обоснование выбора клиент-серверной архитектуры

Для разработки приложения «Интерактивный конструктор образовательных тестов» выбрана клиент-серверная архитектура, позволяющая разделить интерфейс, бизнес-логику и хранение данных.

Клиентская часть отвечает за отображение интерфейса и взаимодействие с пользователем, серверная - за обработку данных, авторизацию, проверку прав доступа и работу с базой данных, которая обеспечивает долговременное хранение информации.

Данный подход обоснован необходимостью поддержки различных ролей пользователей с разными правами доступа и позволяет централизованно контролировать доступ к данным.

Кроме того, клиент-серверная архитектура удобна для развёртывания в контейнеризированной среде, так как компоненты системы могут быть размещены в отдельных Docker-контейнерах.

3.2. Общая структура приложения

Разрабатываемое приложение состоит из следующих основных компонентов:

- React SPA - клиентское одностраничное приложение;
- REST API - серверный программный интерфейс;
- PostgreSQL - база данных;
- Nginx - веб-сервер и reverse proxy;
- Docker-контейнеры - изолированная среда запуска компонентов.

Клиентская часть реализована как SPA-приложение. Пользователь работает с интерфейсом в браузере, а переходы между страницами выполняются без полной перезагрузки сайта. В приложении предусмотрены страницы главной, списка тестов, прохождения теста, входа, регистрации, профиля и конструктора тестов.

Серверная часть предоставляет REST API, через который клиент

получает и отправляет данные. API используется для регистрации и входа пользователей, получения списка тестов, прохождения тестов, создания вопросов и сохранения результатов.

База данных PostgreSQL хранит основные сущности приложения: пользователей, тесты, вопросы, варианты ответов и результаты прохождения. Это позволяет сохранять данные между запусками приложения и обеспечивать совместную работу нескольких пользователей.

Nginx используется для обработки входящих HTTP-запросов. Он может отдавать статические файлы клиентского приложения и перенаправлять API-запросы к backend-сервису.

В проекте используется Docker Compose-конфигурация, в которой выделены сервисы frontend, backend, postgres и nginx, а также общая Docker-сеть для их взаимодействия.

Общая архитектура приложения может быть представлена следующим образом:

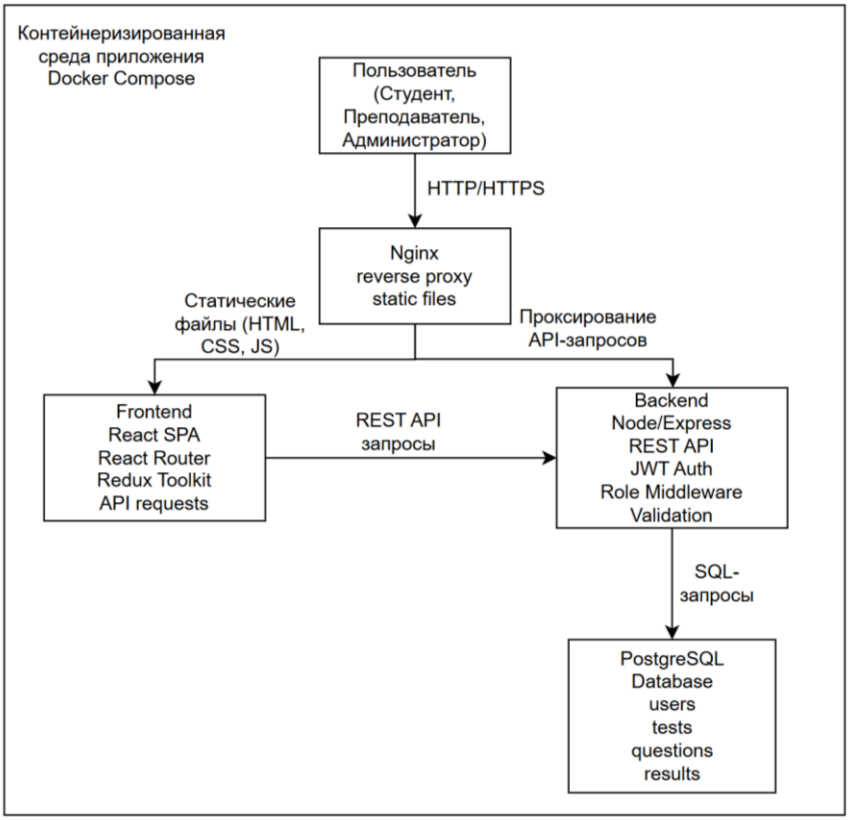

Рисунок 2 - Общая архитектура приложения «Интерактивный конструктор образовательных тестов»

3.3. Взаимодействие компонентов

Взаимодействие компонентов начинается с действий пользователя в браузере. Пользователь открывает приложение, выполняет регистрацию или вход, выбирает тест, проходит его либо переходит в конструктор тестов, если обладает ролью преподавателя или администратора.

Клиентская часть отправляет HTTP-запросы к серверному API. Например, при авторизации пользователь вводит email и пароль, после чего frontend передаёт эти данные на сервер. Сервер проверяет данные пользователя в базе данных и при успешной проверке возвращает токен доступа. Далее клиентская часть использует этот токен при обращении к защищённым маршрутам.

При прохождении теста клиентская часть получает список вопросов от сервера, отображает их пользователю и отправляет ответы на проверку. После завершения тестирования сервер рассчитывает результат и сохраняет его в базе данных.

При работе с конструктором тестов преподаватель может создавать тесты, добавлять вопросы, изменять и удалять их. Все эти действия реализуются через CRUD-операции серверного API. Сервер проверяет роль пользователя и разрешает выполнение операций только пользователям с правами преподавателя или администратора.

Взаимодействие компонентов можно описать следующим образом:

1. Пользователь выполняет действие в интерфейсе.
2. React SPA формирует HTTP-запрос.
3. Nginx принимает запрос и перенаправляет его к backend.
4. Backend проверяет токен и роль пользователя.
5. Backend выполняет бизнес-логику.
6. Backend обращается к PostgreSQL.
7. PostgreSQL возвращает данные.
8. Backend формирует ответ.
9. React SPA отображает результат пользователю.

Компонентную структуру приложения целесообразно представить с помощью UML-диаграммы компонентов.

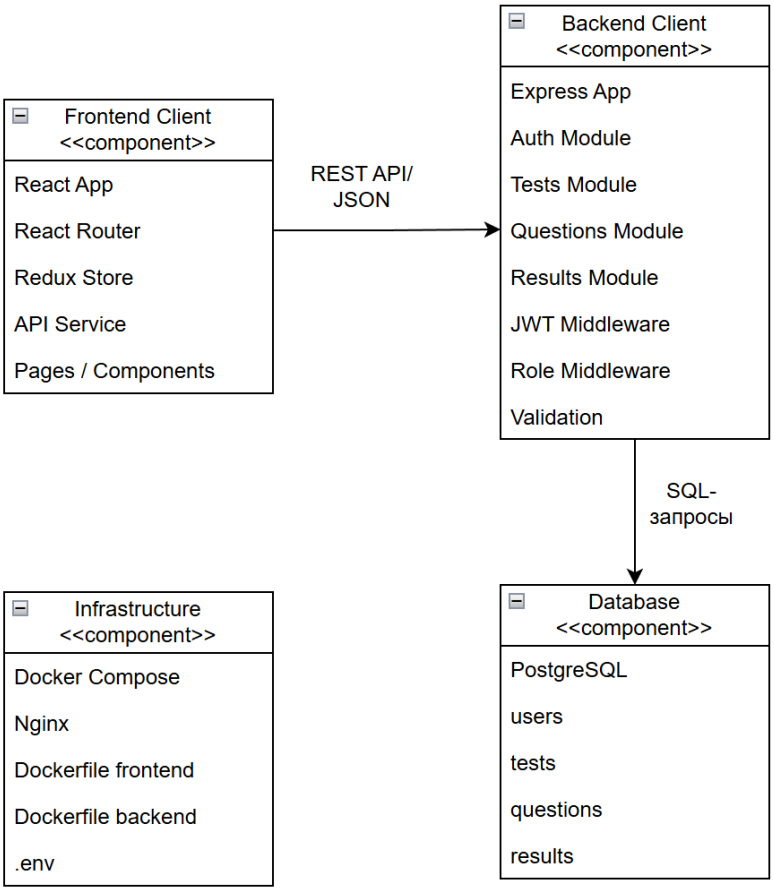

Рисунок 3 - UML-диаграмма компонентов приложения

3.4. Применение методологии двенадцати факторов

При разработке приложения учитываются принципы методологии двенадцати факторов. Данная методология применяется для создания переносимых, масштабируемых и удобных для развёртывания веб-приложений.

В рамках проекта особенно важны следующие принципы:

Кодовая база. Исходный код приложения хранится в системе контроля версий Git. Это позволяет отслеживать изменения, возвращаться к предыдущим версиям и организовать работу над проектом.

Зависимости. Зависимости клиентской и серверной частей должны быть явно описаны в файлах проекта. Это обеспечивает воспроизводимую установку приложения на другой машине или сервере.

Конфигурация. Настройки приложения выносятся во внешние переменные окружения. К таким настройкам относятся параметры подключения к базе данных, JWT-секрет, режим запуска и адрес API. В проекте используются .env-файлы с параметрами POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD, JWT_SECRET, NODE_ENV и REACT_APP_API_URL.

Сторонние службы. База данных PostgreSQL рассматривается как подключаемый ресурс. Серверная часть взаимодействует с ней через строку подключения, что позволяет при необходимости заменить локальную базу данных на облачную.

Процессы. Клиентская часть, серверная часть, база данных и Nginx запускаются как отдельные процессы в отдельных контейнерах. Такой подход повышает изолированность компонентов и упрощает развёртывание.

Сборка, релиз и запуск. Dockerfile и Docker Compose позволяют отделить этап сборки приложения от этапа его запуска. Это делает развёртывание более предсказуемым.

Порт привязки. Backend-сервис запускается на отдельном порту, а Nginx принимает внешние запросы и перенаправляет их к нужному компоненту.

Паритет сред. Использование Docker позволяет приблизить локальную среду разработки к среде развёртывания. Это снижает вероятность ошибок, возникающих из-за различий между окружениями.

Таким образом, выбранная архитектура соответствует требованиям курсовой работы и позволяет реализовать приложение как полноценную клиент-серверную систему с разделением ответственности между компонентами, поддержкой авторизации, ролевой модели, базы данных и контейнерного развёртывания.

4. Выбор программного стека

Для реализации клиент-серверного приложения «Интерактивный конструктор образовательных тестов» был выбран современный стек технологий, обеспечивающий разработку полноценного fullstack CRUD-приложения. Выбранные технологии позволяют реализовать интерактивный интерфейс, серверную бизнес-логику, хранение данных, а также обеспечить удобное развёртывание приложения.

4.1. Frontend: React, React Router, Redux Toolkit, CSS Modules

Клиентская часть приложения реализована с использованием библиотеки React, которая позволяет создавать интерактивные пользовательские интерфейсы на основе компонентного подхода. Такой подход удобен для разработки сложных интерфейсов, состоящих из независимых частей, например: страницы тестов, формы авторизации, конструктора вопросов и интерфейса прохождения теста.

Для организации навигации используется React Router, который обеспечивает переход между страницами без перезагрузки браузера. В приложении реализованы маршруты для главной страницы, списка тестов, страницы прохождения теста, регистрации, авторизации и конструктора.

Управление состоянием приложения осуществляется с помощью Redux Toolkit. Он используется для хранения данных пользователя, токена авторизации, списка тестов, вопросов и результатов прохождения. Централизованное хранилище упрощает передачу данных между компонентами.

Стилизация интерфейса выполняется с использованием CSS Modules, что позволяет изолировать стили компонентов и избежать конфликтов имён классов.

4.2. Backend: Node.js, Express.js

Серверная часть приложения реализована на платформе **Node.js**, которая позволяет использовать язык JavaScript как на клиенте, так и на

сервере. Это упрощает разработку и снижает порог вхождения.

В качестве веб-фреймворка используется **Express.js**. Он предоставляет удобные инструменты для создания REST API, обработки HTTP-запросов и организации middleware.

Серверная часть выполняет следующие функции:

- обработка запросов от клиента;
- реализация CRUD-операций;
- проверка авторизации пользователя;
- разграничение доступа по ролям;
- взаимодействие с базой данных.

4.3. База данных: PostgreSQL

В качестве системы управления базами данных выбрана PostgreSQL. Она обеспечивает надёжное хранение данных и поддерживает работу с реляционными структурами.

В базе данных хранятся:

- пользователи;
- тесты;
- вопросы;
- ответы;
- результаты прохождения.

PostgreSQL позволяет организовать связи между таблицами и обеспечивает целостность данных, что особенно важно для образовательных систем.

4.4. Инфраструктура: Docker, Docker Compose, Nginx

Для контейнеризации приложения используется Docker. Он позволяет упаковать каждую часть системы в отдельный контейнер, что обеспечивает изолированность и переносимость.

Для управления несколькими контейнерами применяется Docker Compose. Он позволяет запускать все компоненты системы (frontend, backend,

база данных, nginx) одной командой.

В качестве веб-сервера и reverse proxy используется Nginx. Он выполняет следующие функции:

- принимает входящие HTTP-запросы;
- отдаёт статические файлы frontend;
- перенаправляет API-запросы на backend.

В проекте реализована конфигурация с сервисами frontend, backend, postgres и nginx, объединёнными в одну сеть.

4.5. Система контроля версий: Git

Для управления исходным кодом используется система контроля версий Git. Она позволяет:

- отслеживать изменения в коде;
- работать с версиями проекта;
- выполнять совместную разработку;
- хранить историю изменений.

Проект организован в виде репозитория с разделением на клиентскую и серверную части, а также конфигурационные файлы для Docker и Nginx.

4.6. Обоснование выбора технологий

Выбранный стек технологий полностью соответствует требованиям к разработке fullstack CRUD-приложения.

React обеспечивает создание динамического пользовательского интерфейса, удобного для работы с тестами. Redux Toolkit упрощает управление состоянием и синхронизацию данных между компонентами.

Node.js и Express.js позволяют быстро реализовать серверную часть с REST API и поддержкой middleware. PostgreSQL обеспечивает надёжное хранение данных и поддержку связей между сущностями.

Docker и Docker Compose позволяют легко развернуть приложение в любой среде, а Nginx обеспечивает эффективную обработку HTTP-запросов.

Использование Git делает процесс разработки управляемым и

воспроизводимым.

Таким образом, выбранный стек технологий является оптимальным для реализации интерактивного конструктора тестов и соответствует современным требованиям веб-разработки.

Таблица 5 - Используемый программный стек

Категория	Технология	Назначение	Категория
Frontend	React	Построение пользовательского интерфейса	Frontend
Frontend	React Router	Маршрутизация страниц	Frontend
Frontend	Redux Toolkit	Управление состоянием	Frontend
Frontend	CSS Modules	Стилизация компонентов	Frontend
Backend	Node.js	Среда выполнения сервера	Backend
Backend	Express.js	Реализация REST API	Backend
База данных	PostgreSQL	Хранение данных	База данных
Инфраструктура	Docker	Контейнеризация	Инфраструктура
Инфраструктура	Docker Compose	Оркестрация контейнеров	Инфраструктура
Инфраструктура	Nginx	Reverse proxy и веб-сервер	Инфраструктура
Контроль версий	Git	Управление исходным кодом	Контроль версий

5. Проектирование базы данных

База данных приложения «Интерактивный конструктор образовательных тестов» предназначена для хранения пользователей, тестов, вопросов, ответов пользователей и результатов прохождения. В проекте используется PostgreSQL, подключение к которому выполняется серверной частью через переменную окружения DATABASE_URL; при запуске backend синхронизирует модели с базой данных через Sequelize.

5.1. Основные сущности

В рамках проектирования базы данных выделены следующие основные сущности:

Таблица 6 – Сущности БД

Категория	Технология
users	Хранение данных пользователей системы
tests	Хранение информации об образовательных тестах
questions	Хранение вопросов, вариантов ответов и правильных ответов
results	Хранение результатов прохождения тестов
answers	Хранение ответов пользователя на вопросы

Сущность users содержит информацию о пользователях приложения. Для каждого пользователя хранятся идентификатор, имя, email, пароль в защищённом виде и роль. Роль пользователя определяет доступные действия в системе: студент может проходить тесты, преподаватель и администратор могут создавать и редактировать тесты.

Сущность tests используется для хранения тестов. Каждый тест имеет название, описание и связь с пользователем, который его создал.

Сущность questions хранит вопросы конкретного теста. Вопрос содержит текст задания, тип вопроса, варианты ответов и правильный ответ. В проекте на стороне клиента при создании вопроса передаются поля question, type, options, left, right и correct, что позволяет поддерживать разные форматы вопросов.

Сущность results хранит итог прохождения теста пользователем: идентификатор пользователя, идентификатор теста, количество правильных

ответов, общее количество вопросов и дату завершения. В проекте результат создаётся после оценки ответов и включает userId, testId, score, totalQuestions и answers.

Сущность answers может использоваться для отдельного хранения ответов пользователя по каждому вопросу. В текущей логике проекта ответы также могут сохраняться внутри результата в JSON-структуре, однако для ER-диаграммы целесообразно выделить answers как отдельную сущность, так как она отражает связь между результатом, вопросом и конкретным ответом пользователя.

5.2. Связи между сущностями

Между сущностями базы данных устанавливаются следующие связи:

Таблица 7 – Связи между сущностями

Связь	Тип связи	Описание
users -> tests	один ко многим	Один преподаватель может создать несколько тестов
tests -> questions	один ко многим	Один тест содержит несколько вопросов
users -> results	один ко многим	Один пользователь может иметь несколько результатов
tests -> results	один ко многим	Один тест может быть пройден разными пользователями
results -> answers	один ко многим	Один результат содержит ответы на несколько вопросов
questions -> answers	один ко многим	Один вопрос может встречаться в разных прохождениях

Такая структура позволяет хранить как сами тестовые материалы, так и историю прохождения тестов пользователями. Например, при удалении теста связанные с ним вопросы и результаты также могут быть удалены каскадно, чтобы в базе данных не оставались лишние записи.

Связь tests и questions является ключевой для конструктора тестов, так как преподаватель сначала создаёт тест, а затем добавляет в него набор вопросов. В пользовательском интерфейсе это выражается через выбор теста и последующее добавление, редактирование или удаление вопросов.

5.3. Хранение вопросов разных типов

В приложении предусмотрена поддержка нескольких типов вопросов:

Таблица 8 – Особенности хранения правильных ответов

Тип вопроса	Назначение	Особенности хранения
single	Один правильный ответ	В options хранится массив вариантов, в correct - индекс правильного варианта
multiple	Несколько правильных ответов	В options хранится массив вариантов, в correct — массив индексов правильных вариантов
matching	Сопоставление элементов	В left_items и right_items хранятся части пар, в correct — соответствия между ними

Для вопросов типа single пользователь выбирает один вариант ответа.

Для вопросов типа multiple пользователь может выбрать несколько вариантов.

Для вопросов типа matching используются два массива: левая и правая части сопоставления. В проекте при создании matching-вопроса проверяется

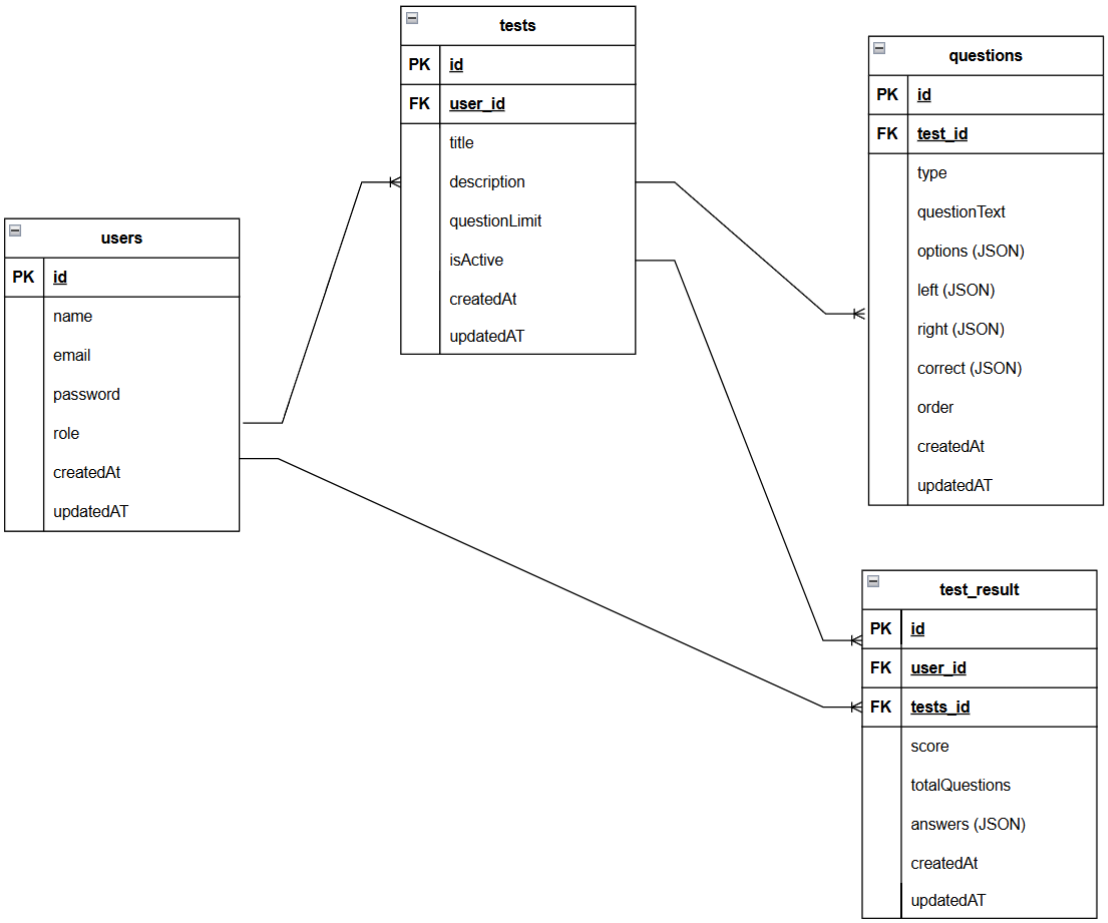

Рисунок 5 — ER-диаграмма базы данных приложения «Интерактивный конструктор образовательных тестов»

6. Разработка клиентской части

Клиентская часть приложения «Интерактивный конструктор образовательных тестов» реализована как одностраничное React-приложение. Она отвечает за отображение интерфейса, маршрутизацию между страницами, взаимодействие с REST API, хранение состояния пользователя и обработку сценариев прохождения тестов. В проекте точкой входа является client/src/index.js, где React-приложение монтируется в DOM-элемент root, а основной компонент приложения подключается через App.

6.1. Структура React-приложения

Код клиентской части расположен в директории client. Основная логика приложения находится в каталоге client/src, где выделены страницы, переиспользуемые компоненты, Redux-хранилище, API-сервисы и файлы стилей.

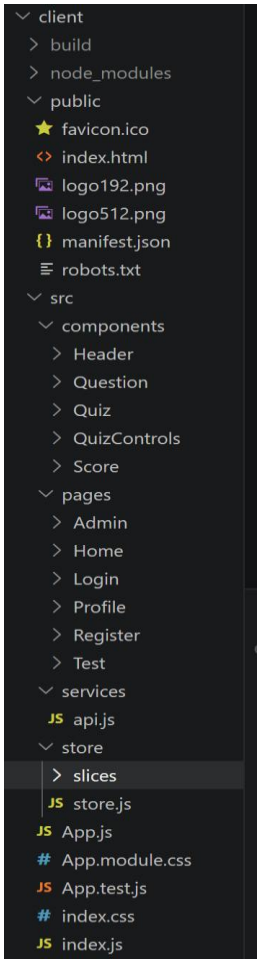

Рисунок 6 – Структура клиентской части

Компонентный подход позволяет разделить интерфейс на независимые части. Например, шапка сайта реализована в компоненте Header, отображение вопроса — в Question, управление прохождением теста — в QuizContainer, а элементы навигации по вопросам — в QuizControls.

6.2. Маршрутизация страниц

Маршрутизация реализована с помощью react-router-dom. В файле App.js используется BrowserRouter, внутри которого определены маршруты ко всем основным страницам приложения. Также всё приложение обёрнуто в Provider, что обеспечивает доступ компонентов к Redux-хранилищу.

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/tests" element={<Test />} />
  <Route path="/test/:testId" element={<QuizContainer />} />
  <Route path="/login" element={<Login />} />
  <Route path="/register" element={<Register />} />
  <Route path="/profile" element={<Profile />} />
  <Route path="/admin/questions" element={<QuestionsAdmin />} />
  <Route path="/constructor" element={<QuestionsAdmin />} />
</Routes>
```

Рисунок 7 – Маршруты

Маршрут /tests используется для отображения списка тестов. Маршрут /test/:testId открывает страницу прохождения конкретного теста по его идентификатору. Маршруты /admin/questions и /constructor ведут к интерфейсу конструктора вопросов.

При загрузке приложения выполняется проверка состояния авторизации. Если в состоянии есть токен, но данные пользователя ещё не загружены, вызывается действие fetchProfile, которое получает профиль пользователя с сервера.

6.3. Главная страница

Главная страница является стартовым экраном приложения. Она предназначена для краткого представления системы пользователю и перехода

к основным сценариям работы: просмотру тестов, регистрации, входу или использованию конструктора при наличии соответствующей роли.

Главная страница должна выполнять навигационную и информационную функцию. Она объясняет назначение приложения и помогает пользователю выбрать дальнейшее действие.

6.4. Страница тестов

Страница тестов отображает список доступных образовательных тестов. Каждый тест представлен карточкой с названием, описанием, количеством вопросов и кнопкой перехода к прохождению. В коде карточка содержит ссылку вида /test/${test.id}, что позволяет открыть конкретный тест по его идентификатору.

Если список тестов пуст, пользователь видит соответствующее сообщение. Такое поведение необходимо для корректной обработки ситуации, когда тестовые данные ещё не добавлены или сервер не вернул доступные тесты.

6.5. Компонент прохождения теста

Прохождение теста реализовано в компоненте QuizContainer. Он получает testId из параметров маршрута, загружает вопросы, управляет текущим вопросом, сохраняет ответы пользователя и отправляет результат после завершения тестирования. Для отображения интерфейса внутри него используются компоненты Score, Question и QuizControls.

Компонент Score показывает прогресс прохождения. Если пользователь проходит обычный тест, отображается строка вида «Пройдено вопросов: текущий / всего». Если включён режим повтора ошибок, отображается количество ошибочных вопросов.

Компонент Question отвечает за отображение вопроса. Он поддерживает обычные вопросы с одним или несколькими вариантами ответа, а также вопросы типа matching, где пользователь сопоставляет элементы через выпадающие списки. В компоненте предусмотрена проверка ответа и

отображение таблицы обратной связи с правильными и неправильными вариантами.

После завершения теста приложение отображает количество правильных ответов, процент выполнения, текстовую оценку результата, кнопку повторного прохождения и возможность повторить только ошибочные вопросы.

6.6. Конструктор вопросов

Конструктор вопросов реализован в компоненте QuestionsAdmin. Он предназначен для пользователей с ролью преподавателя или администратора. Если пользователь не обладает нужной ролью, компонент отображает сообщение о запрете доступа.

В конструкторе доступны следующие действия:

- создание нового теста;
- выбор существующего теста;
- добавление вопроса;
- редактирование вопроса;
- удаление вопроса;
- удаление выбранного теста.

Конструктор поддерживает три типа вопросов: single, multiple и matching. Для вопросов single используется радиокнопка, для multiple — чекбоксы, а для matching — пары полей «левая часть» и «правая часть» с выбором соответствия. Перед отправкой на сервер данные вопроса проходят подготовку в функции buildQuestionPayload: пустые варианты отбрасываются, проверяется минимальное количество вариантов и наличие правильного ответа. Для matching-вопроса проверяется заполнение всех пар и наличие соответствия для каждой строки.

6.7. Работа с состоянием через Redux

Для управления состоянием используется Redux Toolkit. Redux-хранилище подключается в корневом компоненте через Provider, а компоненты получают

доступ к данным с помощью useSelector и изменяют состояние через useDispatch.

В клиентской части состояние используется для нескольких ключевых задач:

authSlice: хранение токена, хранение данных пользователя, проверка профиля, выход из системы.

quizSlice: хранение списка вопросов, хранение номера текущего вопроса, сохранение ответов пользователя, отправка результата, режим повтора ошибок.

Для связи с backend используется файл services/api.js. В нём создаётся экземпляр axios с базовым адресом из переменной окружения REACT_APP_API_URL. Также настроен interceptor, который добавляет JWT-токен из localStorage в заголовок Authorization при каждом запросе.

Такой механизм позволяет автоматически отправлять токен при обращении к защищённым API-маршрутам и не дублировать эту логику в каждом компоненте.

7. Разработка серверной части

Серверная часть приложения «Интерактивный конструктор образовательных тестов» реализована на Node.js с использованием Express.js. Она отвечает за обработку HTTP-запросов от клиентского приложения, выполнение бизнес-логики, авторизацию пользователей, проверку ролей, валидацию входных данных и взаимодействие с базой данных PostgreSQL. В проекте backend запускается как отдельный сервис backend, получает параметры подключения через переменные окружения и работает с базой данных через DATABASE_URL.

7.1. Структура backend-приложения

Серверная часть расположена в директории server. Внутри неё выделены отдельные модули для маршрутов, контроллеров, моделей базы данных, middleware и конфигурации подключения к базе данных.

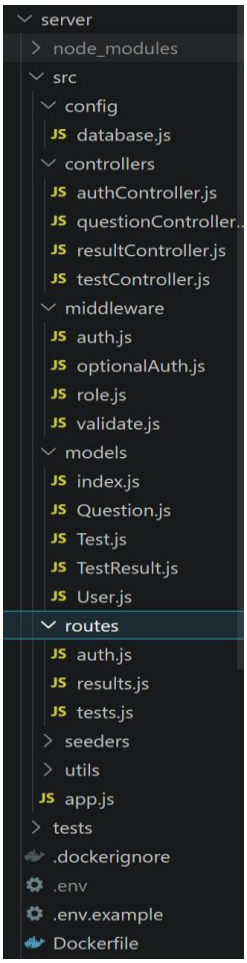

Рисунок 8 – Структура backend-приложения

Такое разделение позволяет отделить описание маршрутов от бизнес-логики и моделей данных. Например, маршруты отвечают за приём HTTP-запросов, контроллеры — за выполнение операций, middleware — за проверку авторизации, ролей и входных данных, а модели описывают структуру таблиц базы данных.

7.2. REST API

Серверная часть предоставляет REST API, через который клиентское приложение выполняет регистрацию, вход, получение профиля, загрузку тестов, управление вопросами и сохранение результатов.

API условно разделён на несколько групп:

/api/auth — регистрация, вход, профиль пользователя;

/api/tests — получение тестов, создание, изменение и удаление тестов;

/api/results — получение и сохранение результатов прохождения.

В маршрутах авторизации предусмотрены запросы для регистрации, входа и получения профиля пользователя. Для регистрации проверяется корректность email, минимальная длина пароля, длина имени и ограничение публичной регистрации только ролью student.

7.3. Реализация CRUD-операций

CRUD-операции реализованы для тестов и вопросов. Пользователь с ролью преподавателя или администратора может создавать тесты, изменять их, удалять, а также добавлять, редактировать и удалять вопросы.

В маршрутах тестов используются следующие операции:

- GET /api/tests
- GET /api/tests/:id
- POST /api/tests
- PUT /api/tests/:id
- DELETE /api/tests/:id

Для вопросов используются вложенные маршруты:

- GET /api/tests/:id/questions

- GET /api/tests/:id/questions/manage
- POST /api/tests/:id/questions
- PUT /api/tests/:id/questions/:questionId
- DELETE /api/tests/:id/questions/:questionId

В проекте маршруты создания, редактирования и удаления вопросов защищены middleware авторизации и проверки ролей teacher и admin. Также для параметров id и questionId используется проверка, что они являются положительными целыми числами.

7.4. Авторизация и аутентификация через JWT

Аутентификация пользователя выполняется через email и пароль. При регистрации пароль не сохраняется в открытом виде: в модели User используется хеширование с помощью bcryptjs перед созданием или обновлением записи. Для проверки пароля реализован метод comparePassword, а при преобразовании пользователя в JSON пароль удаляется из результата.

После успешного входа сервер формирует JWT-токен. Клиентская часть сохраняет его и передаёт в заголовке Authorization при обращении к защищённым маршрутам.

Общая логика авторизации выглядит следующим образом:

1. Пользователь вводит email и пароль.
2. Frontend отправляет данные на /api/auth/login.
3. Backend ищет пользователя в базе данных.
4. Backend сравнивает пароль с хешем.
5. При успешной проверке сервер формирует JWT.
6. Frontend сохраняет токен и использует его в следующих запросах.

Для получения профиля используется защищённый маршрут /profile, который доступен только после прохождения middleware auth.

7.5. Проверка ролей пользователей

В приложении реализована ролевая модель. Пользователь может иметь

одну из трёх ролей:

- student - прохождение тестов;
- teacher - создание и управление тестами;
- admin - полный доступ к функциям управления.

В модели пользователя поле role описано как перечисление student, teacher, admin, а роль по умолчанию устанавливается как student.

Проверка ролей используется для ограничения доступа к административным действиям. Например, создание тестов, удаление тестов, получение вопросов для управления и изменение вопросов доступны только пользователям с ролью teacher или admin.

Такой подход защищает серверную часть от прямого обращения к API пользователем без нужных прав, даже если он вручную отправит HTTP-запрос.

7.6. Работа с PostgreSQL

Для работы с PostgreSQL используется Sequelize. В проекте определены модели User, Test, Question и TestResult. Модель Question поддерживает типы вопросов single, multiple и matching, а варианты ответов, пары сопоставления и правильные ответы хранятся в JSON-полях options, left, right и correct.

Модель Test содержит название, описание, лимит вопросов, автора, признак активности и дату создания. Модель TestResult хранит пользователя, тест, количество правильных ответов, общее количество вопросов, ответы и дату завершения.

Связи между моделями позволяют описать предметную область приложения:

- User -> Test один пользователь может создать несколько тестов;
- Test -> Question один тест содержит несколько вопросов;
- User -> TestResult один пользователь может иметь несколько результатов;
- Test -> TestResult один тест может иметь несколько результатов прохождения.

Работа с PostgreSQL обеспечивает сохранение данных между запусками приложения и позволяет хранить как структуру тестов, так и результаты прохождения пользователями.

Таблица 9 — Основные API-маршруты

Метод	Endpoint	Назначение	Доступ
POST	/api/auth/register	Регистрация пользователя	Публичный
POST	/api/auth/login	Вход пользователя	Публичный
GET	/api/auth/profile	Получение профиля	Авторизованный пользователь
GET	/api/tests	Получение списка тестов	Публичный
GET	/api/tests/:id	Получение данных теста	Публичный
GET	/api/tests/:id/questions	Получение вопросов для прохождения	Публичный
POST	/api/tests/:id/submit	Отправка ответов и расчёт результата	Публичный / авторизованный
POST	/api/tests	Создание теста	Преподаватель, администратор
PUT	/api/tests/:id	Редактирование теста	Авторизованный пользователь
DELETE	/api/tests/:id	Удаление теста	Преподаватель, администратор
GET	/api/tests/:id/questions/manage	Получение вопросов для конструктора	Преподаватель, администратор
POST	/api/tests/:id/questions	Добавление вопроса	Преподаватель, администратор
PUT	/api/tests/:id/questions/:questionId	Редактирование вопроса	Преподаватель, администратор
DELETE	/api/tests/:id/questions/:questionId	Удаление вопроса	Преподаватель, администратор
GET	/api/results/my	Получение результатов пользователя	Авторизованный пользователь

8. Тестирование приложения

Тестирование приложения «Интерактивный конструктор образовательных тестов» проводилось для проверки корректности основных пользовательских сценариев, работы клиентской и серверной частей, обработки ошибок, ролевой модели и устойчивости API к некорректным данным.

В проекте уже присутствуют компонентные тесты клиентской части. Например, в App.test.js проверяется отображение главной страницы конструктора и работа формы добавления вариантов ответа в конструкторе вопросов. Тест имитирует пользователя с ролью teacher, открывает конструктор, добавляет вариант ответа, удаляет варианты и проверяет, что минимальное количество вариантов не становится меньше двух.

8.1. Регистрация и авторизация

Первым этапом тестирования была проверена регистрация нового пользователя и последующая авторизация в системе. Данный сценарий является базовым, так как доступ к профилю, сохранению результатов и конструктору зависит от успешного входа пользователя.

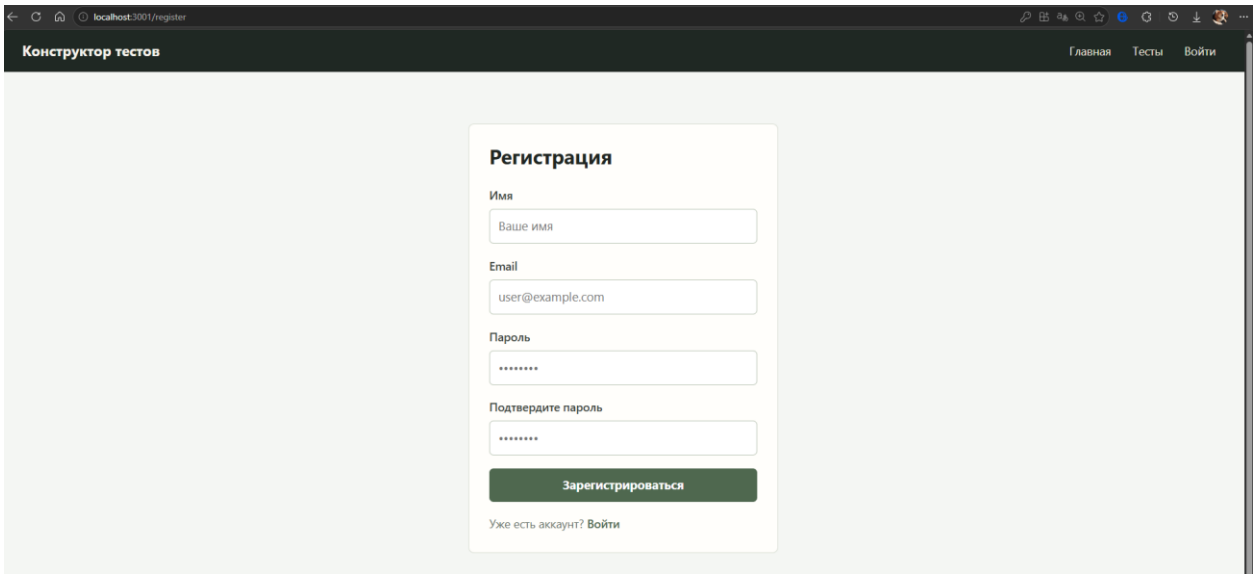

Рисунок 9 - Форма регистрации пользователя

После заполнения формы регистрации система должна создать пользователя с ролью student по умолчанию и отобразить успешный результат

регистрации.

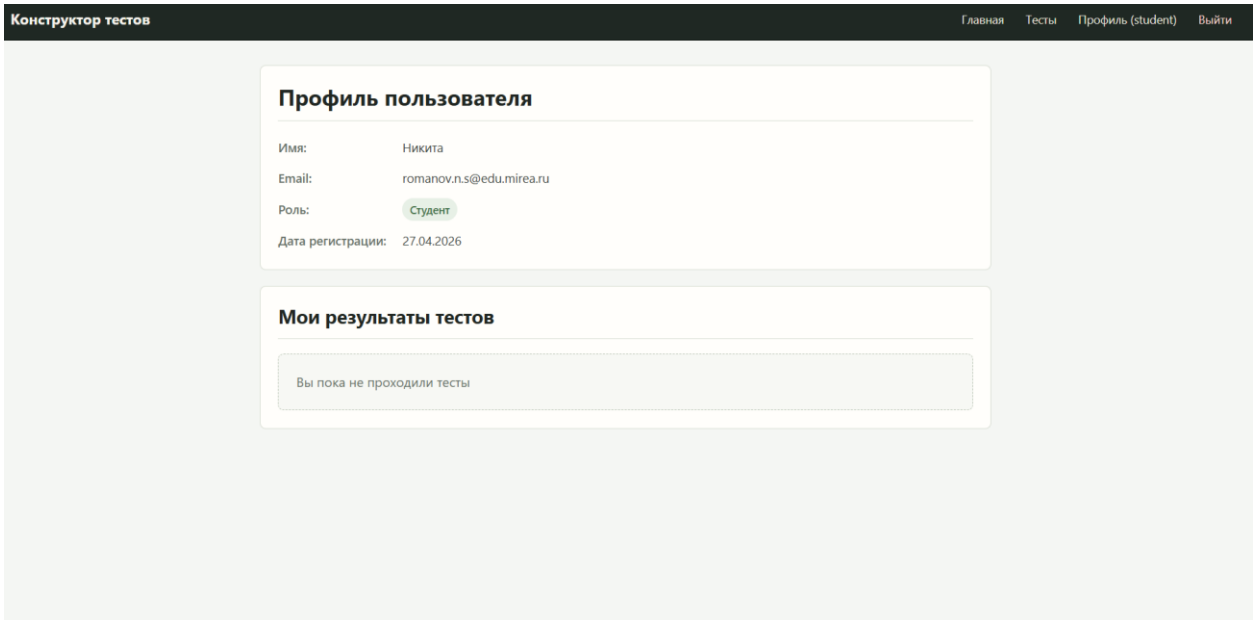

Рисунок 10 - Успешная регистрация пользователя

Далее выполнялась проверка авторизации. Пользователь вводит email и пароль, после чего сервер проверяет данные и возвращает JWT-токен. После успешного входа пользователь получает доступ к профилю и основным функциям приложения.

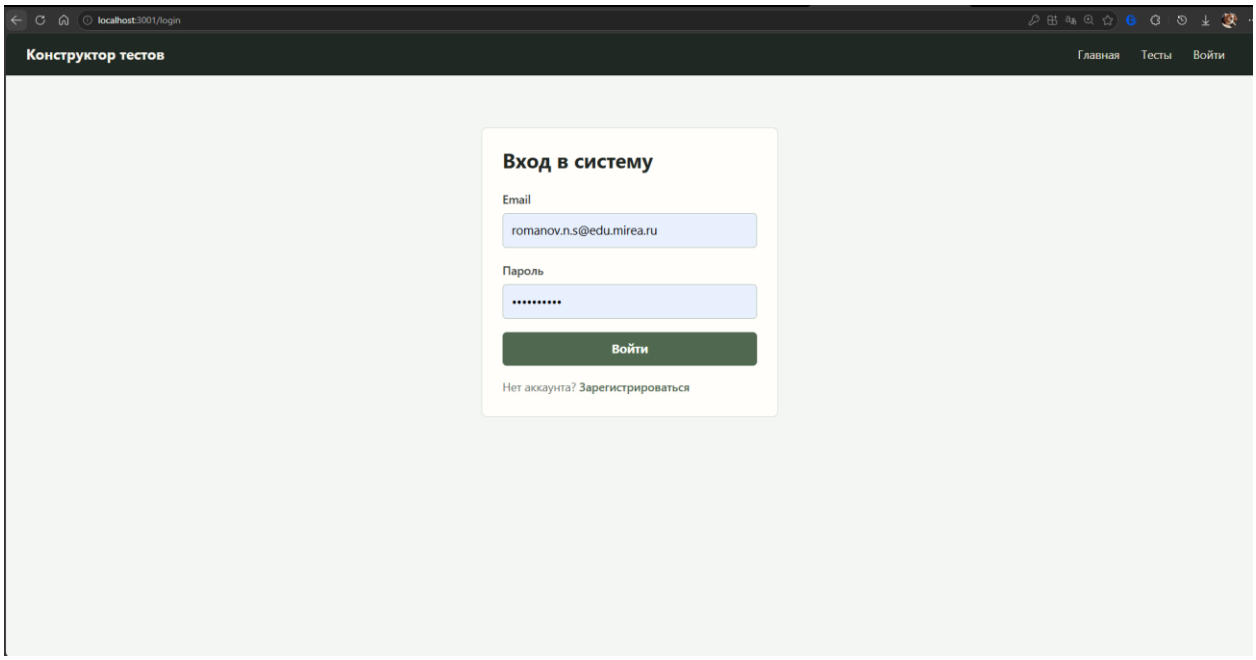

Рисунок 11 - Форма авторизации пользователя

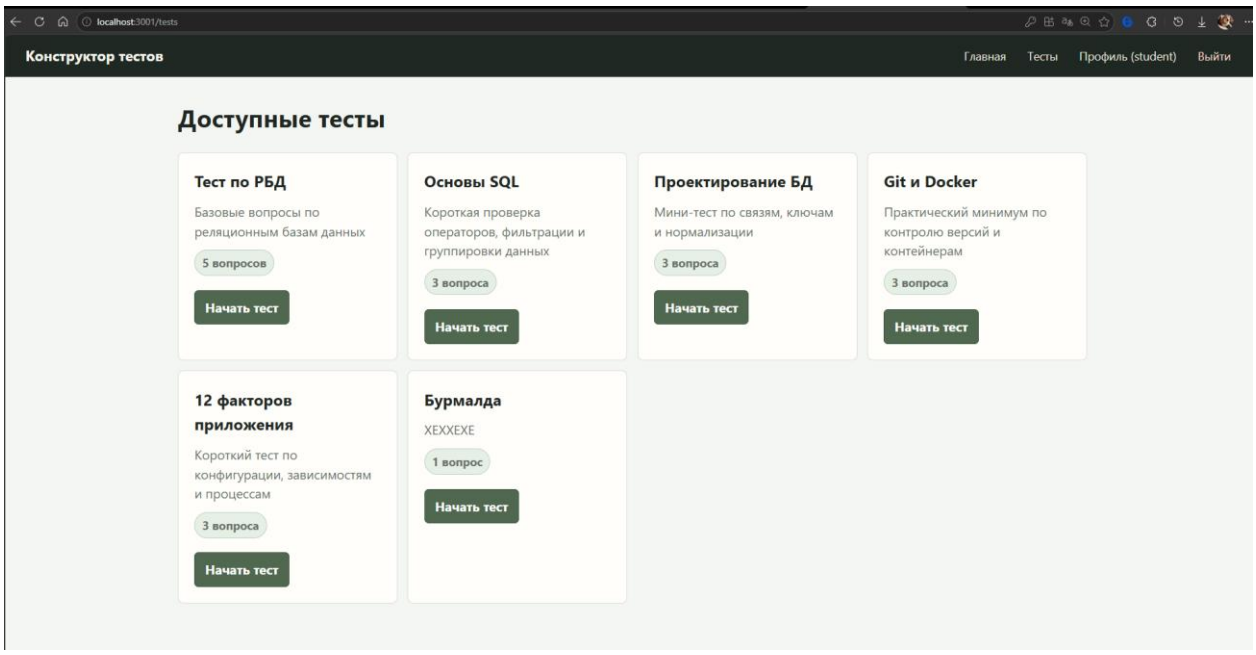

Рисунок 12 - Успешная авторизация пользователя

8.2. Проверка основных пользовательских сценариев

После проверки регистрации и входа были протестированы основные сценарии работы приложения.

8.2.1 Просмотр списка тестов

Пользователь открывает страницу со списком тестов. Система отправляет запрос к серверу и отображает доступные тесты. Для каждого теста выводится название, описание и кнопка перехода к прохождению.

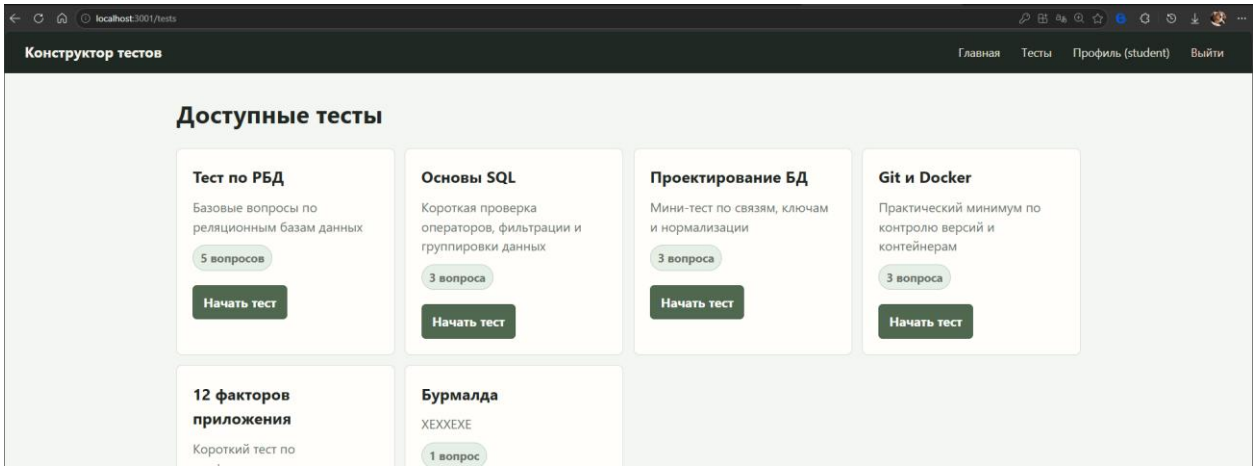

Рисунок 13 - Список доступных тестов

8.2.2 Прохождение теста

Пользователь выбирает тест и переходит на страницу прохождения.

Компонент QuizContainer загружает вопросы, отображает текущий вопрос, сохраняет ответы пользователя и отправляет результат после завершения тестирования. В Redux-состоянии сохраняются текущий вопрос, ответы пользователя, итоговый результат и список ошибочных вопросов.

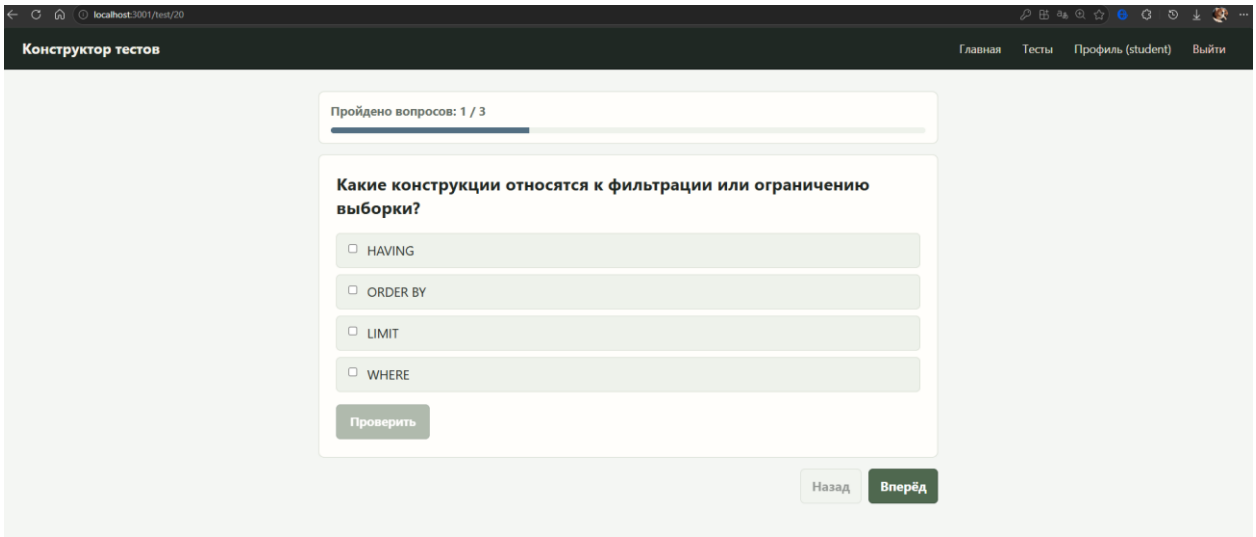

Рисунок 14 - Прохождение теста

После завершения теста приложение отображает количество правильных ответов, процент выполнения и текстовую оценку результата.

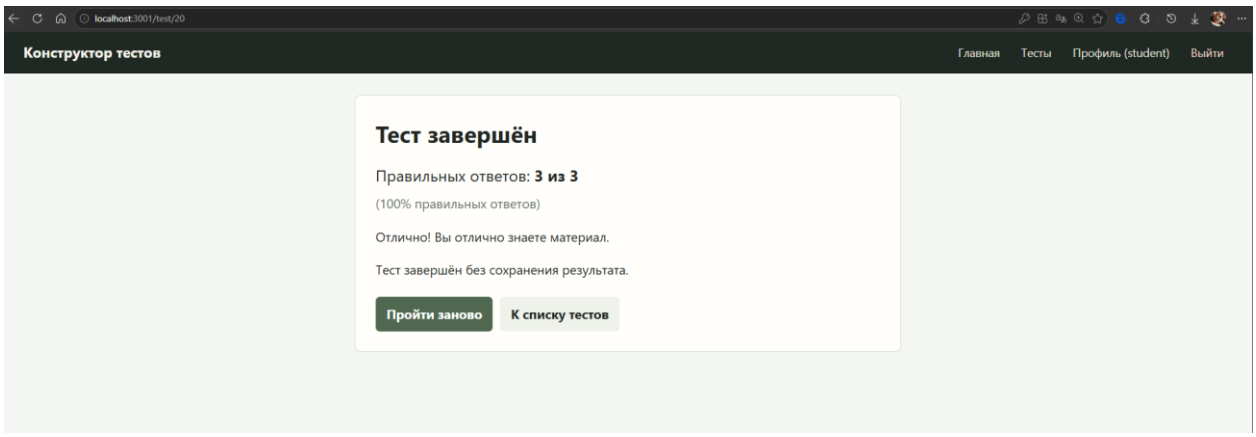

Рисунок 15 — Результат прохождения теста

8.2.3 Повтор ошибок

Если после завершения теста есть неправильные ответы, пользователь может повторить только ошибочные вопросы. Данный сценарий проверяет работу режима wrong, который используется для повторного прохождения вопросов, на которые пользователь ответил неверно.

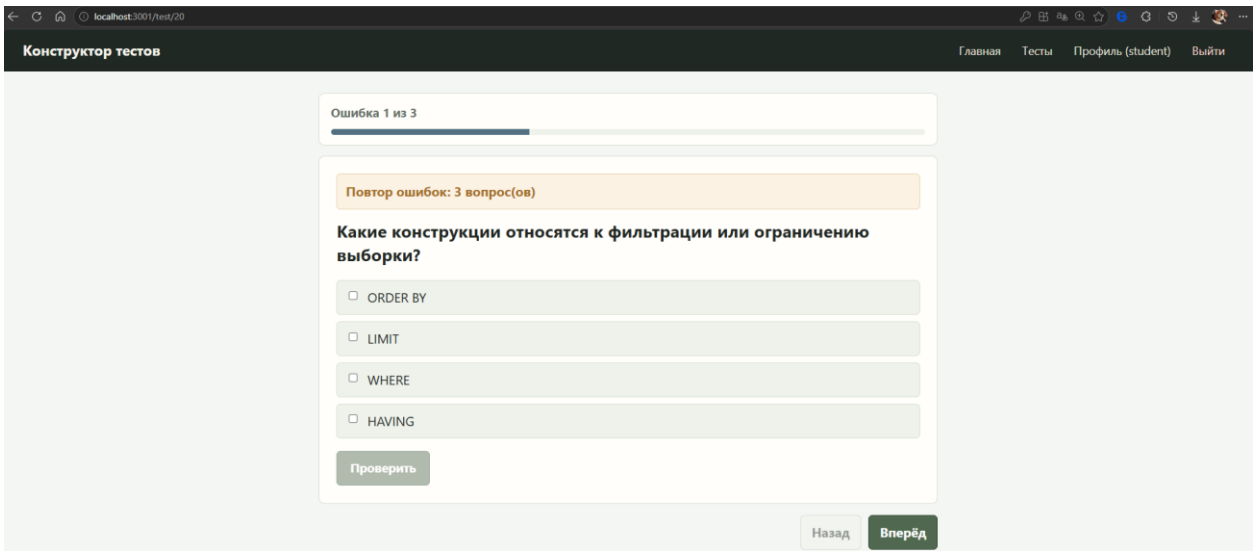

Рисунок 16 — Повтор ошибочных вопросов

8.2.4 Создание теста

Для пользователя с ролью teacher или admin проверялась возможность открытия конструктора и создания нового теста. В форме указывается название и описание теста.

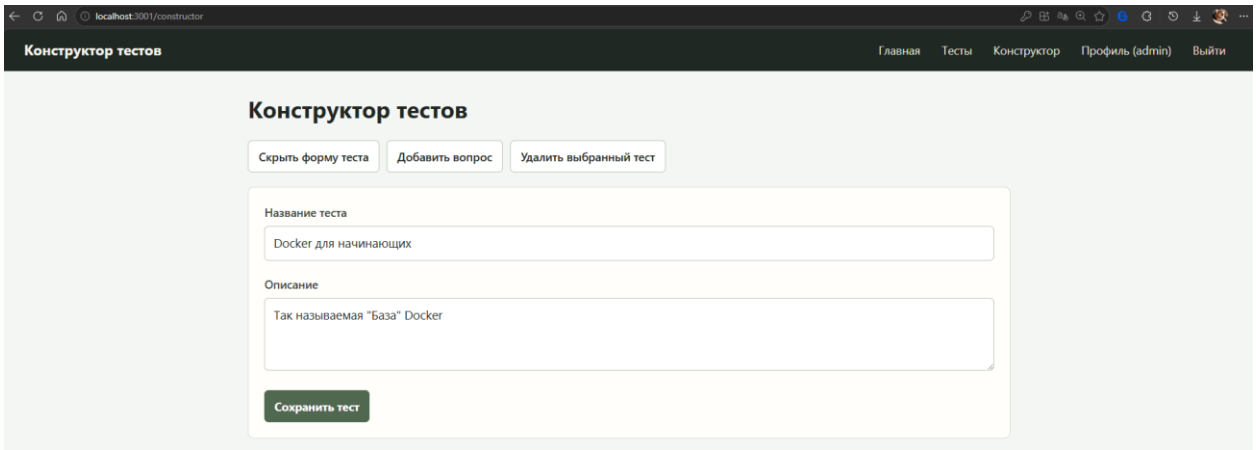

Рисунок 17 — Создание нового теста

8.2.5 Добавление вопроса

В конструкторе проверялась возможность добавления вопросов разных типов:

- вопрос с одним правильным ответом;
- вопрос с несколькими правильными ответами;
- вопрос на сопоставление.

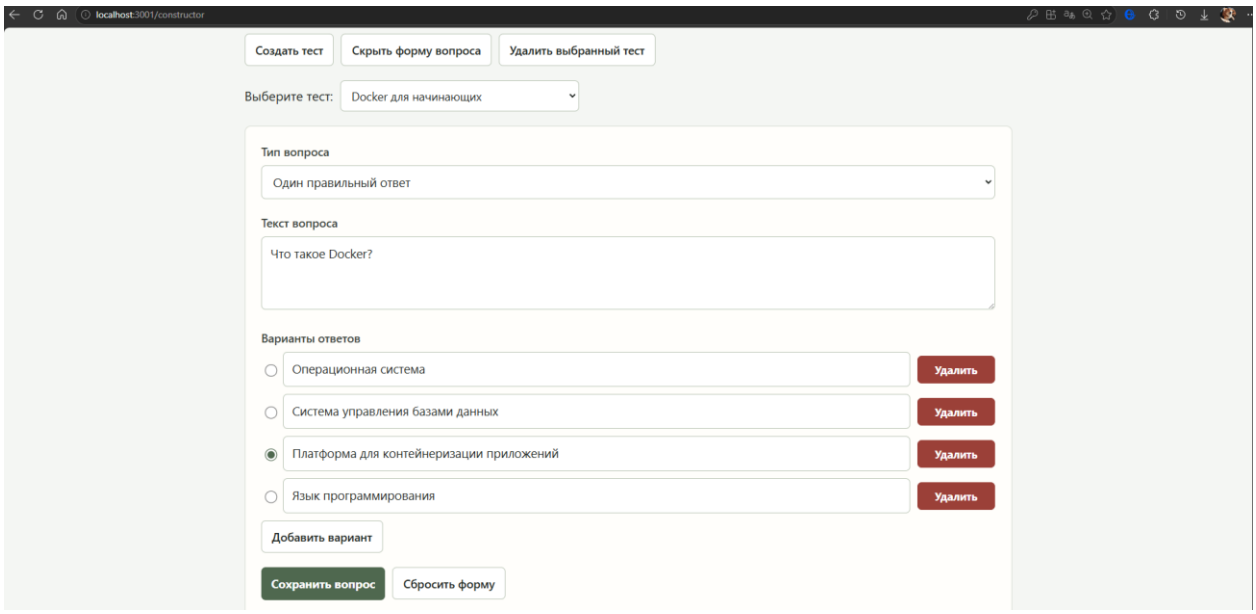

Рисунок 18 — Добавление вопроса с одним правильным ответом

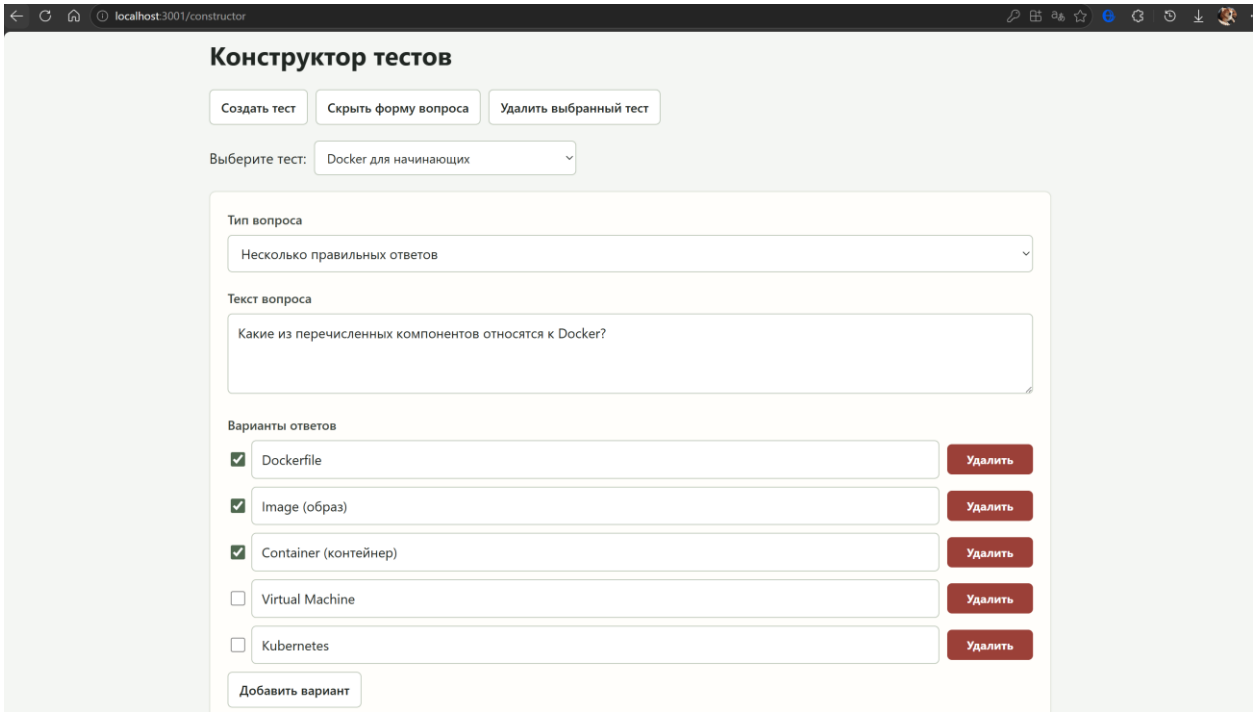

Рисунок 19 — Добавление вопроса с несколькими правильными ответами

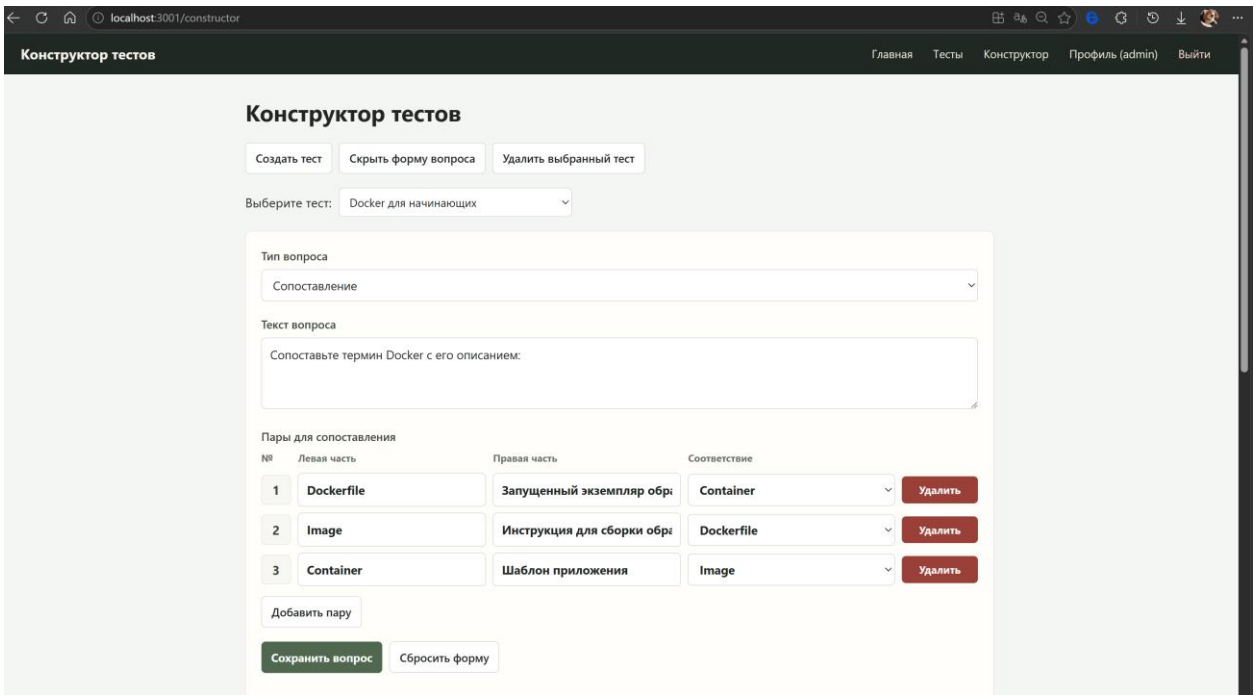

Рисунок 20 — Добавление вопроса на сопоставление

8.3. Модульное и компонентное тестирование

Для проверки клиентской части используется Jest и React Testing Library. Тесты позволяют проверить работу компонентов без ручного запуска всего приложения.

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP\client> npm test
> test-constructor-client@1.0.0 test
> react-scripts test
 PASS  src/App.test.js
  √ renders constructor home page (109 ms)
  √ allows removing answer options in the question constructor (292 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        4.679 s
Ran all test suites related to changed files.

Watch Usage
 › Press a to run all tests.
 › Press f to run only failed tests.
 › Press q to quit watch mode.
 › Press p to filter by a filename regex pattern.
 › Press t to filter by a test name regex pattern.
 › Press Enter to trigger a test run.
```

Рисунок 21 — Успешное выполнение компонентных тестов

8.4. Фаззинг-тестирование API

Фаззинг-тестирование проводилось для проверки устойчивости серверной части к некорректным, случайным и повреждённым входным данным. Целью тестирования было убедиться, что сервер не завершает работу аварийно и возвращает корректные HTTP-ответы с ошибками.

Проверялись следующие группы некорректных данных:

- неверный формат email;
- слишком короткий пароль;
- попытка регистрации с ролью admin;
- пустое название теста;
- неправильный тип вопроса;
- пустой массив вариантов ответа;
- повреждённый JSON;
- неверный JWT-токен;
- отрицательные или строковые идентификаторы тестов и вопросов.

```
PS C:\Users\relax\Documents\kursach_6_PiRKSP\server\tests\fuzzing> npm test

Testing XSS payload handling...

PASS XSS Register Payload -> /api/auth/register (201)
PASS XSS Register Payload -> /api/auth/register (201)
PASS XSS Register Payload -> /api/auth/register (201)

Testing authentication guards...

PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)
PASS Protected Profile Access -> /api/auth/profile (401)

Testing path traversal, command injection and null bytes...

PASS Traversal/Command Route Probe -> /api/tests/..%2F..%2F..%2Fetc%2Fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/..%5C..%5C..%5Cwindows%5Csystem32%5Cconfig%5Csam/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%252e%252e%252f%252e%252e%252f%252e%252e%252fetc%252fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%3B%20cat%20%2Fetc%2Fpasswd/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%26%26%20whoami/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%24(id)/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%00/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/test%00admin/questions (400)
PASS Traversal/Command Route Probe -> /api/tests/%2500/questions (400)

Testing large payloads...

PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)
PASS Large Payload Register -> /api/auth/register (400)

Testing RBAC...

PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS Auth Login -> /api/auth/login (200)
PASS RBAC Create Test (admin) -> /api/tests (201)
PASS RBAC Delete Test (admin) -> /api/tests/38 (200)
PASS RBAC Create Test (teacher) -> /api/tests (201)
PASS RBAC Delete Test (teacher) -> /api/tests/39 (200)
PASS RBAC Create Test (student) -> /api/tests (403)

Results saved to: C:\Users\relax\Documents\kursach_6_PiRKSP\server\tests\fuzzing\results/fuzzing-report-1777241872641.json
Passed checks: 37
Failed checks: 0
```

Рисунок 22 — Успешное выполнение фаззинг-тестирования API

9. Размещение проекта в Git и контейнеризация

В рамках выполнения курсовой работы разработанное клиент-серверное приложение было оформлено в виде проекта и размещено в системе контроля версий Git.

Исходный код проекта размещён в удалённом репозитории:
Ссылка на репозиторий: https://github.com/Relax1205/kursach_6_PiRKSP
В процессе работы использовались возможности Git для:

- хранения и версионирования исходного кода;
- ведения истории изменений;
- организации структуры проекта;
- подготовки проекта к развёртыванию.

10. Контейнеризация и развёртывание

Для упрощения развертывания приложения используется технология Docker. Все компоненты системы запускаются в отдельных контейнерах и объединяются в одну сеть.

10.1. Dockerfile клиентской части

Клиентская часть собирается в два этапа:

1. Сборка React-приложения;
2. Размещение собранных файлов в контейнере Nginx.

```
client > Dockerfile
1   FROM node:18-alpine AS build
2
3   WORKDIR /app
4
5   COPY package*.json ./
6   RUN npm ci
7
8   COPY . .
9
10  ARG REACT_APP_API_URL
11  ENV REACT_APP_API_URL=$REACT_APP_API_URL
12
13  RUN npm run build
14
15  FROM nginx:alpine
16
17  COPY nginx.conf /etc/nginx/conf.d/default.conf
18
19  COPY --from=build /app/build /usr/share/nginx/html
```

Рисунок 23 – содержимое Docker файла клиентской части

Такой подход уменьшает размер финального образа и ускоряет загрузку приложения.

10.2. Dockerfile серверной части

Серверная часть запускается в контейнере Node.js.

Основные этапы:

- установка зависимостей;

- копирование исходного кода;
- запуск приложения.

```
Dockerfile
server > Dockerfile
1   FROM node:18-alpine
2
3   WORKDIR /app
4
5   COPY package*.json ./
6
7   RUN npm ci --only=production
8
9   COPY . .
10
11
12  CMD ["node", "src/app.js"]
13
```

Рисунок 24 – Содержимое Docker файла серверной части

10.3. Docker Compose

Для управления контейнерами используется Docker Compose.

В проекте определены следующие сервисы:

- postgres — база данных;
- backend — серверное приложение;
- frontend — клиентская часть;
- nginx — reverse proxy.

Контейнеры объединены в одну сеть app-network, что позволяет им взаимодействовать по внутренним именам.

10.4. PostgreSQL-контейнер

данных запускается в отдельном контейнере:

- используется образ postgres:15-alpine;
- данные сохраняются в volume postgres_data;
- выполняется автоматическая инициализация базы.

Подключение к базе осуществляется через переменную окружения:

DATABASE_URL=postgresql://user:password@postgres:5432/testdb

10.5. Nginx как reverse proxy

Nginx выполняет роль промежуточного сервера:

- принимает HTTP-запросы;
- направляет API-запросы к backend;
- отдаёт статические файлы frontend.

10.6. Развёртывание в облаке

Для развёртывания приложения используется Docker Compose.

Основные шаги:

docker-compose up --build

В production-среде используется файл:

docker-compose.ubuntu.yml

Он включает:

- перезапуск контейнеров (restart: unless-stopped);
- проверку готовности базы данных (healthcheck);
- изоляцию сети.

После запуска приложение доступно через браузер по адресу сервера.

Заключение

В рамках выполнения курсовой работы была разработана система «Интерактивный конструктор образовательных тестов», представляющая собой клиент-серверное веб-приложение для создания и прохождения тестов.

В ходе работы была подтверждена актуальность выбранной темы, обусловленная широким использованием цифровых образовательных технологий и необходимостью автоматизации контроля знаний обучающихся. Разработка подобных систем позволяет повысить объективность оценки, сократить время проверки и обеспечить удобный доступ к образовательным материалам.

В процессе выполнения работы были решены все поставленные задачи. Проведён анализ предметной области, определены пользователи системы и основные сценарии её использования. Были сформулированы функциональные и нефункциональные требования, а также требования к безопасности и ролевой модели доступа.

Разработана архитектура приложения на основе клиент-серверного подхода с использованием React SPA, REST API и базы данных PostgreSQL. Проведено проектирование базы данных, определены основные сущности и связи между ними, реализована поддержка различных типов вопросов.

В рамках практической реализации разработаны клиентская и серверная части приложения. Клиентская часть обеспечивает удобный пользовательский интерфейс, маршрутизацию страниц и управление состоянием с помощью Redux. Серверная часть реализует REST API, обработку запросов, авторизацию пользователей с использованием JWT, проверку ролей и взаимодействие с базой данных.

Реализована ролевая модель доступа, обеспечивающая разграничение прав пользователей. Проведена валидация входных данных как на клиентской, так и на серверной стороне. Это позволило повысить устойчивость системы к некорректным и потенциально опасным запросам.

Проведено тестирование приложения, включая проверку основных

пользовательских сценариев, компонентное тестирование клиентской части и фаззинг-тестирование API. Результаты тестирования показали корректную работу системы и устойчивость к ошибочным входным данным.

Проект был размещён в системе контроля версий Git, что позволило организовать хранение и управление исходным кодом. Для упрощения развёртывания использованы технологии Docker и Docker Compose, обеспечивающие запуск всех компонентов приложения в контейнерной среде. Также была выполнена настройка Nginx в качестве reverse proxy.

Таким образом, разработанное приложение соответствует поставленной цели и требованиям курсовой работы, а также демонстрирует практическое применение современных технологий веб-разработки.

Список используемой литературы

1. Фаулер М. Архитектура корпоративных программных приложений. — Москва: Вильямс, 2019. — 544 с.
2. Richards M., Ford N. Fundamentals of Software Architecture: An Engineering Approach. — Sebastopol: O'Reilly Media, 2020. — 432 p.
3. Fielding R. Architectural Styles and the Design of Network-based Software Architectures: Doctoral dissertation. — Irvine: University of California, 2000. — 162 p.
4. The Twelve-Factor App [Электронный ресурс]. — Режим доступа: https://12factor.net/ (дата обращения: 26.04.2026).
5. React — A JavaScript library for building user interfaces [Электронный ресурс]. — Режим доступа: https://react.dev/ (дата обращения: 26.04.2026).
6. React Router Documentation [Электронный ресурс]. — Режим доступа: https://reactrouter.com/ (дата обращения: 26.04.2026).
7. Redux Toolkit Documentation [Электронный ресурс]. — Режим доступа: https://redux-toolkit.js.org/ (дата обращения: 26.04.2026).
8. Node.js Documentation [Электронный ресурс]. — Режим доступа: https://nodejs.org/ (дата обращения: 26.04.2026).
9. Express.js Documentation [Электронный ресурс]. — Режим доступа: https://expressjs.com/ (дата обращения: 26.04.2026).
10. PostgreSQL Documentation [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 26.04.2026).
11. Docker Documentation [Электронный ресурс]. — Режим доступа: https://docs.docker.com/ (дата обращения: 26.04.2026).
12. Интерактивный конструктор образовательных тестов [Электронный ресурс]: репозиторий. — Режим доступа: https://github.com/Relax1205/kursach_6_PiRKSP(дата обращения: 26.04.2026).